

## TD 10 — Supply chain et sécurité

### Objectifs

- Analyser des incidents de sécurité réels
  - Comprendre et créer un SBOM
  - Évaluer la sécurité d'un projet open source
- 

### Exercice 1 — Analyse de Log4Shell

#### Contexte

En décembre 2021, la vulnérabilité CVE-2021-44228 (Log4Shell) a été découverte dans Log4j, une bibliothèque de logging Java extrêmement répandue.

#### Documents à consulter

- CVE : <https://nvd.nist.gov/vuln/detail/CVE-2021-44228>
- Analyse technique : <https://www.lunasec.io/docs/blog/log4j-zero-day/>

#### Questions

1. **Qu'est-ce que Log4j ?** Pourquoi est-il si répandu ?
2. **Comment fonctionne la vulnérabilité ?** Expliquez le mécanisme JNDI lookup.
3. **Pourquoi le score CVSS est-il de 10.0 ?** Quels critères justifient ce score maximal ?
4. **Combien de temps s'est écoulé** entre la découverte et le patch ?
5. **Pourquoi certaines organisations ont mis des semaines à patcher ?** Identifiez les obstacles.

#### Réflexion

Si vous étiez responsable sécurité d'une entreprise utilisant Log4j :

1. Comment auriez-vous su que vous étiez affecté ?
  2. Quelles mesures immédiates auriez-vous prises ?
  3. Quelles mesures préventives auriez-vous mises en place après coup ?
- 

### Exercice 2 — L'attaque XZ Utils

#### Contexte

En mars 2024, une porte dérobée sophistiquée a été découverte dans XZ Utils, une bibliothèque de compression utilisée par SSH.

#### Documents à consulter

- Thread original : <https://www.openwall.com/lists/oss-security/2024/03/29/4>
- Analyse : <https://gist.github.com/thesamesam/223949d5a074ebc3dce9ee78baad9e27>

#### Questions de compréhension

1. **Qui est "Jia Tan" ?** Quelle était sa stratégie ?
2. **Comment la backdoor a-t-elle été découverte ?** Par qui et comment ?
3. **Pourquoi le mainteneur original a-t-il accepté ce nouveau contributeur ?**
4. **Quelles distributions étaient affectées ?** Pourquoi pas toutes ?

#### Analyse comparative

| Critère        | Log4Shell | XZ Utils |
|----------------|-----------|----------|
| Type d'attaque |           |          |

|                         |           |          |
|-------------------------|-----------|----------|
| Critère                 | Log4Shell | XZ Utils |
| Vecteur                 |           |          |
| Temps de préparation    |           |          |
| Méthode de découverte   |           |          |
| Facilité d'exploitation |           |          |
| Leçons principales      |           |          |

## Débat

Comment prévenir ce type d'attaque "social engineering" à long terme sur un projet open source ?

---

## Exercice 3 — Création d'un SBOM

### Instructions

Pour un projet de votre choix (ou le projet fourni), générez et analysez un SBOM.

#### Option A : Projet existant

Clonez un petit projet open source et générez son SBOM :

```
# Installer Syft
curl -sSfL https://raw.githubusercontent.com/anchore/syft/main/install.sh | sh -s --
-b /usr/local/bin

# Générer un SBOM
syft dir:./mon-projet -o cyclonedx-json > sbom.json

# Ou avec Trivy
trivy fs --format cyclonedx ./mon-projet > sbom.json
```

#### Option B : Projet fictif

Créez manuellement un SBOM CycloneDX pour une application fictive avec ces dépendances :

- Flask 2.3.0
- SQLAlchemy 2.0.15
- requests 2.31.0
- python-dotenv 1.0.0

### Questions d'analyse

1. **Combien de composants** contient votre SBOM (directs + transitifs) ?
2. **Quelles licences** sont présentes ? Y a-t-il des conflits potentiels ?
3. **Identifiez 3 dépendances transitives** que vous ne connaissiez pas.
4. **Y a-t-il des vulnérabilités connues** ? Utilisez :

```
grype sbom:./sbom.json
```

### Livrable

Produisez un résumé de votre SBOM :

Projet : \_\_\_\_\_

Composants directs : \_\_\_\_

Composants transitifs : \_\_\_\_

Licences uniques : \_\_\_\_  
Vulnérabilités trouvées : \_\_\_\_

---

## Exercice 4 — Évaluation avec OpenSSF Scorecard

### Instructions

Utilisez OpenSSF Scorecard pour évaluer la sécurité de projets open source.

### Projets à évaluer

Choisissez 2 projets parmi :

- <https://github.com/kubernetes/kubernetes>
- <https://github.com/facebook/react>
- <https://github.com/pallets/flask>
- <https://github.com/expressjs/express>

### Utilisation de Scorecard

```
# Via l'API (pas d'installation)
curl "https://api.securityscorecards.dev/projects/github.com/pallets/flask"
```

Ou via le site : <https://securityscorecards.dev/>

### Grille d'évaluation

Pour chaque projet, notez :

| Critère           | Projet 1 | Projet 2 |
|-------------------|----------|----------|
| Score global      | /10      | /10      |
| Code-Review       |          |          |
| Maintained        |          |          |
| Vulnerabilities   |          |          |
| Branch-Protection |          |          |
| Signed-Releases   |          |          |

### Analyse

1. Quel projet a le meilleur score ? Pourquoi ?
  2. Quels critères sont les plus faibles pour chaque projet ?
  3. Utiliseriez-vous ces projets dans une application critique ? Justifiez.
- 

## Exercice 5 — Plan de réponse aux incidents (si temps restant)

### Scénario

Vous êtes développeur dans une startup. Un lundi matin, vous apprenez qu'une vulnérabilité critique (CVSS 9.8) a été publiée pour une de vos dépendances directes.

### Questions à traiter

1. **Premières actions (30 min)**
  - Comment confirmez-vous que vous êtes affecté ?
  - Qui prévenez-vous ?
  - Quelles mesures immédiates prenez-vous ?
2. **Actions à court terme (24h)**

- Comment identifiez-vous tous les systèmes affectés ?
- Quel est votre plan de mise à jour ?
- Comment communiquez-vous en interne/externe ?

### 3. Actions à moyen terme (1 semaine)

- Quelles mesures préventives mettez-vous en place ?
- Comment améliorez-vous votre processus ?

## Template de plan de réponse

### ## Plan de Réponse - Vulnérabilité Critique

#### ### Phase 1 : Détection et confirmation

- [ ] \_\_\_\_\_
- [ ] \_\_\_\_\_

#### ### Phase 2 : Containment

- [ ] \_\_\_\_\_
- [ ] \_\_\_\_\_

#### ### Phase 3 : Remediation

- [ ] \_\_\_\_\_
- [ ] \_\_\_\_\_

#### ### Phase 4 : Recovery

- [ ] \_\_\_\_\_
- [ ] \_\_\_\_\_

#### ### Phase 5 : Lessons Learned

- [ ] \_\_\_\_\_
- [ ] \_\_\_\_\_

## Exercice 6 — Quiz sécurité supply chain (bonus)

### Instructions

Pour chaque affirmation, indiquez si elle est vraie ou fausse et justifiez.

1. “Un SBOM garantit qu’un logiciel est sécurisé.”

→ \_\_\_\_\_

Justification : \_\_\_\_\_

1. “Les dépendances transitives représentent un risque plus faible que les dépendances directes.”

→ \_\_\_\_\_

Justification : \_\_\_\_\_

1. “Un projet avec beaucoup d’étoiles GitHub est nécessairement bien maintenu.”

→ \_\_\_\_\_

Justification : \_\_\_\_\_

1. “Le typosquatting ne concerne que npm, pas PyPI ou Maven.”

→ \_\_\_\_\_

Justification : \_\_\_\_\_

1. “Verrouiller les versions de dépendances élimine tous les risques supply chain.”

→ \_\_\_\_\_

Justification : \_\_\_\_\_

---

## Pour aller plus loin

### Lectures

- **“Securing the Software Supply Chain”** – CISA/NSA guidelines
- **“Reflections on Trusting Trust”** – Ken Thompson (1984)
- Blog post original sur XZ : <https://www.openwall.com/lists/oss-security/2024/03/29/4>

### Ressources

- **OpenSSF** – <https://openssf.org/>
- **SLSA** – <https://slsa.dev/>
- **Sigstore** – <https://sigstore.dev/>
- **OSV Database** – <https://osv.dev/>

### Outils à explorer

- **Syft** – Génération de SBOM
- **Grype** – Scan de vulnérabilités
- **Trivy** – Scanner tout-en-un
- **Dependabot** – Mises à jour automatiques

### Préparation examen (séance 11)

1. Revoir les concepts clés des séances 1 à 10
2. Préparer une fiche de synthèse personnelle
3. Revoir les TD et corrigés